

rauc

Helix OTA — Stack Research Note: RAUC (Robust Auto-Update Controller)

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Evidence-based evaluation of RAUC as a candidate wrapped update engine for the future Helix Linux phase. Covers A/B bundle format, X.509/CMS verification, HTTP streaming, the D-Bus control surface, and bootloader support. RAUC is an embedded-Linux engine; it is not an Android client and is out of scope for the Android 15 first phase. It is assessed here as a foundation-corpus input for the eventual universal/Linux phase.
Issues	A few claims could not be pinned to an exact primary-source string within the time budget and are tagged “UNVERIFIED — needs confirmation” inline (notably: precise streaming benchmark hardware/figure, exact crypt AES mode, and the first-commit date). Do not promote these to spec without confirmation.
Issues summary	No fabricated numbers; every star/date/figure either cites a consulted URL or is flagged unverified.
Fixed	initial research pass
Fixed summary	Primary sources are the official RAUC ReadTheDocs (basic/advanced/using) and the upstream GitHub repo;

Field	Value
Continuation	secondary cross-checks from comparison write-ups are labelled as such. Feed into research/ota_landscape_report.md and the engine-selection ADR (ADR-0001). When the Linux phase is scheduled, deepen with a hands-on spike (build a verity bundle, stream it over the Helix Go control plane, exercise the D-Bus InstallBundle/Completed loop). Re-run web verification of version/release facts before any decision is locked.

Table of contents

- [§1. Scope & relevance to Helix](#)
- [§2. What RAUC is \(snapshot\)](#)
- [§3. A/B slot model & bundle format](#)
- [§4. X.509 / CMS signature verification](#)
- [§5. HTTP streaming installation](#)
- [§6. D-Bus control surface](#)
- [§7. Bootloader support](#)
- [§8. Go control-plane fit](#)
- [§9. Android fit](#)
- [§10. Wrappability assessment](#)
- [§11. Pros / cons / risks](#)
- [§12. Recommendation](#)
- [§13. Sources consulted](#)

§1. Scope & relevance to Helix

Helix OTA is **Android 15 first**, using native A/B via Android's update_engine plus a custom Go control plane. RAUC does **not** apply to that phase: it is an embedded-Linux update controller and has no Android client. Its relevance is strictly to the **future Linux / universal phase**, where Helix would need an A/B engine for non-Android Linux targets. In that role RAUC is one of the three dominant open-source options (alongside SWUpdate and Mender) and is the closest conceptual analogue to Android's update_engine: declarative redundant slots, atomic full-partition writes, bootloader-driven rollback, mandatory bundle signing.

This note evaluates RAUC against the Helix design constraints: a custom **Go** control plane, HTTP delivery (Helix mandates Brotli + HTTP/3→HTTP/2 fallback, REST primary), strong supply-chain integrity, and the ability to **wrap** the engine behind an OS-adaptor interface rather than adopt its server stack.

§2. What RAUC is (snapshot)

Attribute	Value	Source confidence
Project	RAUC (“Robust Auto-Update Controller”)	high — official site/repo
Origin	Created by Pengutronix (German embedded-Linux consultancy)	high
First commit	~April 2015 — UNVERIFIED — needs confirmation (seen in a secondary comparison write-up, not confirmed against git log)	low
License	LGPL-2.1-or-later	high — GitHub repo
Language	C (~83%), with Python (test/tooling ~10%), shell	high — GitHub repo
Latest release	v1.15.2, 2026-03-27 (per GitHub at time of fetch); also saw v1.14 “stable” and v1.15.x dev tags in docs	medium — single fetch, re-verify before locking
Binary footprint	“~512 KiB” daemon binary — UNVERIFIED — needs confirmation (secondary source)	low
Core deps	GLib (libglib2.0), OpenSSL (libssl), D-Bus (libdbus-1), libcurl; plus libnl-genl-3 (streaming/NBD) and libjson-glib (JSON)	high — repo build deps
Notable production use	Valve Steam Deck / SteamOS 3.0 (secondary source)	medium

RAUC runs as a system D-Bus service on the target device. It does **not** ship a server/control plane — bundle distribution is left to the integrator, which is exactly what Helix wants (Helix supplies its own Go control plane).

§3. A/B slot model & bundle format

3.1 Slot model

RAUC's model is **declaratively configured redundant slots** grouped into classes (e.g. `rootfs.0` / `rootfs.1`). The default is symmetric A/B (two boot slots), but RAUC supports more than two slots per class, asymmetric setups, and **parent-child slot relationships** so a single bundle can atomically update several related partitions (e.g. `rootfs` + `appfs`) as one unit. The running system writes the update to the inactive slot, then reboots into it. Confirmation is done by marking the booted slot group **good** or **bad** (`rauc status mark-good` / `mark-bad`, also exposed on D-Bus). This is structurally the same idea as Android `update_engine`'s active/inactive slots and `markBootSuccessful`.

3.2 Bundle = signed SquashFS

A RAUC **bundle** packages: the filesystem image(s)/archive(s) to install, a **manifest** (lists images, options, meta-info), and optional install hooks/scripts. These are collected into a **SquashFS** image so the target can mount the bundle directly without unpacking to intermediate storage. The SquashFS is followed by a CMS signature over the full image (see §4).

3.3 Three bundle formats

Source: RAUC `advanced.html` / `basic.html`.

Format	Integrity model	Streaming	Encryption	Notes
plain	CMS signature over the whole SquashFS; verified before install. No per-block authentication during read.	No	No	Original format. Cannot be streamed; needs the full bundle present. Subject to a documented concurrent-modification concern that the verity format avoids.
verity	dm-verity Merkle hash tree over the SquashFS payload → per-block authenticated random access.	Yes	No	Enables HTTP streaming and on-the-fly authentication. Recommended modern default.

Format	Integrity model	Streaming	Encryption	Notes
crypt	Built on the verity format; payload is symmetrically encrypted (AES-256 — mode UNVERIFIED — needs confirmation), manifest re-encrypted per recipient via rauc encrypt.	Yes	Yes	Multi-recipient: any private key matching a recipient certificate can decrypt. For confidential payloads.

For Helix’s streaming + integrity goals, **verity** (or **crypt** if payload confidentiality is required) is the relevant format; plain would not be used.

§4. X.509 / CMS signature verification

This is one of RAUC’s strongest areas and aligns well with Helix’s supply-chain requirements.

- **Signing is mandatory.** There is no unsigned-bundle path; a self-signed cert is permitted only for development.
- **Signature container:** the SquashFS is followed by a signature stored in **CMS (Cryptographic Message Syntax, RFC 5652)** format, including the signer’s certificate. Backed by OpenSSL.
- **Verification flow:** before install, the signer certificate is verified against the on-device **keyring(s)**; the signer’s public key then verifies the bundle signature.
- **Certificate chains / intermediates:** RAUC supports embedding **intermediate certificates** in the bundle signature (`--intermediate`, repeatable) to close the trust chain to a CA anchor in the device keyring. Doc warns intermediates needed to reach the trust anchor should **not** be placed in the keyring itself.
- **Key-usage enforcement:** by default RAUC does **not** check key-usage attributes; optional `check-purpose=codesign` enforces `extendedKeyUsage` containing `codeSigning` on the leaf cert. Helix should enable this in production.
- **HSM / smart-card:** RAUC can use keys/certs on **PKCS#11** tokens (YubiKey, HSM) via RFC 7512 PKCS#11 URLs for `--cert/--key`; PIN via `RAUC_PKCS11_PIN` or prompt. Good for a hardened Helix signing pipeline.
- **SPKI hashes:** RAUC can compute SPKI hashes (over the full public-key info) to compare certificate ownership independent of signature metadata — useful for cert-pinning / rotation logic.

Net: full X.509 PKI with chain building, intermediates, optional codesign EKU enforcement, and HSM support. This is a stronger, more standards-based story than ad-hoc SHA-256 + detached signature.

§5. HTTP streaming installation

Built-in since **v1.6** for the verity and crypt formats (the verity bundle was introduced to enable exactly this).

- **No intermediate storage:** bundle images are streamed directly into the target slot; no temporary copy of the full bundle on device.
- **Mechanism:** RAUC sets up a kernel **NBD (network block device)** plus an unprivileged helper that translates NBD read requests into **HTTP Range Requests** against the server. On top of the NBD device, **dm-verity** authenticates each block on the fly.
- **Server requirement:** the HTTP(S) server must support **HTTP Range Requests**. (This is a hard requirement Helix's Go control plane / object store must satisfy for the streaming path.)
- **Kernel requirement:** target kernel needs **NBD support** enabled.
- **TLS / headers:** streaming installs can pass TLS client cert/key and custom HTTP headers (relevant for authenticating to a Helix endpoint).
- **Adaptive (not delta) updates:** RAUC explicitly distinguishes **adaptive** updates (e.g. block-hash-index) — which inspect existing slot data to skip re-downloading unchanged blocks, transparently, with no bundle-format change — from **delta** updates. casync-based chunked diff delivery also exists but needs a separate server-side chunk store; note the doc warns that enabling streaming support means RAUC can **no longer** download plain casync bundles.
- **Benchmark:** doc cites a streaming install figure ($\approx 1\text{m}43\text{s}$ for a $\sim 190\text{ MiB}$ bundle on an STM32MP1-class board) comparable to download-then-install — **exact figure/hardware UNVERIFIED** — **needs confirmation** (single fetch, treat as indicative only).

For Helix this is the headline feature: a device-pull streaming model over plain HTTP(S) Range Requests fits a custom Go control plane far better than engines that demand their own server protocol. Brotli/HTTP3 specifics would need spike validation (RAUC uses libcurl; Range-request semantics over HTTP/2/3 should hold but is unverified for Helix's exact stack).

§6. D-Bus control surface

RAUC's primary integration API is **D-Bus** (it is “controllable” from a host process):

- **Bus name:** `de.pengutronix.rauc`
- **Object path:** `/`
- **Interface:** `de.pengutronix.rauc.Installer`
- **Method `InstallBundle`:** triggers a background install and returns immediately; accepts arguments incl. `require-manifest-hash`, and streaming options for TLS cert/key and HTTP headers.
- **Signal `Completed`:** emitted on finish (success or failure).
- **Property `Progress`:** updated continuously during install for progress monitoring.
- **Property `LastError`:** last error string.
- **Mark good/bad:** a D-Bus method (plus `rauc status mark-good/mark-bad` CLI) marks the booted slot group confirmed/failed.
- **Inspect:** `InspectBundle` D-Bus method (a `json-2 rauc info` output matches its structure).

This is a clean, async, signal-driven control surface — easy to wrap behind a Helix OS-adaptor: call `InstallBundle`, subscribe to `Progress/Completed`, then `Mark`.

§7. Bootloader support

RAUC delegates the boot-slot switch and rollback-on-failure to the bootloader. Supported out of the box (per repo + docs):

- **U-Boot** (via scripting / env)
- **GRUB** (via scripting / env)
- **Barebox** (via dedicated bootchooser infrastructure)
- **EFI** (listed in repo feature set)
- **Custom** boot-selection implementations

The bootloader is responsible for booting the inactive slot and, on repeated boot failure, falling back — the same division of responsibility Android draws between `update_engine` and the bootloader's slot/retry logic. For a Helix Linux target, the chosen bootloader (commonly U-Boot/GRUB/EFI) must be configured with RAUC's boot-selection contract.

§8. Go control-plane fit

- **No server to fight:** RAUC ships **no** control plane. Distribution, rollout orchestration, telemetry, and staged rollout remain Helix's Go responsibility — a strong fit with the locked "custom Go control plane" decision; Helix would not inherit a competing server stack (contrast with Mender/hawkBit).
- **Delivery contract is plain HTTP(S) + Range Requests** for streaming — implementable directly in Gin over the mandated transport stack (HTTP/3→HTTP/2 fallback; Brotli at the artifact layer would need validation against Range-request/dm-verity block semantics — flag for spike).
- **Device control is D-Bus, not Go-native.** There is a community Go helper, [gitlab.com/zygoon/go-rauc-raucdbus](https://github.com/zygoon/go-rauc-raucdbus) (Apache-2.0, v0.4.0 dated **2023-08-18, community / not official, appears unmaintained**), but it only exposes **constants** for `InstallBundle/Completed/Progress/LastError` — it is **not** a full client and does **not** cover `Mark/GetSlotStatus`. Practically, the on-device agent that talks D-Bus to RAUC is more naturally a small C/Go process on the device using a general D-Bus library (e.g. github.com/godbus/dbus); the *Go control plane* talks HTTP to that agent, not D-Bus directly. Plan to implement the D-Bus binding ourselves rather than depend on the unmaintained helper.

§9. Android fit

None / not applicable. RAUC is an embedded-Linux engine with no Android client, no integration with Android's `update_engine`, recovery, dynamic partitions, or Virtual A/B. For the Android 15 first phase Helix uses Android-native A/B (`update_engine.applyPayload`). RAUC should be explicitly recorded as **out of scope for Android** and reconsidered only when the Linux phase is scheduled.

§10. Wrappability assessment

High. RAUC is essentially built to be wrapped:

- Clear separation of concerns: RAUC = device-side install + slot/rollback; integrator = distribution + orchestration.
- Async D-Bus API (InstallBundle + Progress/Completed + Mark) maps cleanly onto a Helix OS-adapter interface (install / observe-progress / confirm / rollback).
- Streaming pull over standard HTTP(S) Range Requests means the Helix Go control plane only needs to serve verity/crypt bundles from its artifact store with Range support.
- CMS/X.509 signing integrates with an external PKI/HSM, so Helix's signing pipeline stays authoritative.

Main wrapping cost: writing/maintaining the on-device D-Bus glue (no production-grade official Go binding) and configuring the bootloader contract per target.

§11. Pros / cons / risks

Pros - Mature, widely deployed (incl. Steam Deck), focused embedded-Linux A/B engine — direct update_engine analogue for Linux targets. - Strong, standards-based security: mandatory CMS/RFC 5652 signing, X.509 chains + intermediates, optional codeSigning ECU enforcement, PKCS#11/HSM. - Built-in HTTP(S) **streaming** (verity/crypt) with no intermediate storage; per-block dm-verity authentication; adaptive (block-hash-index) updates. - Ships **no** server — does not compete with the Helix Go control plane; permissive-enough LGPL-2.1. - Flexible slot model (multi-slot, parent-child) and broad storage backend support (eMMC, NAND/NOR, UBI/UBIFS). - Clean async D-Bus control surface, easy to wrap.

Cons / risks - **Android: not applicable** — zero value for the first phase. - Device-side is **C + D-Bus + GLib/OpenSSL**, not Go; no maintained official Go client (the one community package is constants-only, unmaintained, 2023). - Streaming needs **kernel NBD** + server **Range Request** support — extra kernel/infra requirements; interaction with Helix's Brotli/HTTP3 choices is **unverified**. - LGPL-2.1 dynamic-linking considerations for any in-process linking (D-Bus/CLI usage avoids this; confirm with the legal lens before bundling). - Several quantitative claims here (binary size, benchmark, first-commit date, AES mode, exact latest version) are flagged unverified and must be confirmed before any ADR locks on them.

§12. Recommendation

Shortlist for the future Helix Linux phase; do not adopt for the Android 15 first phase. RAUC is the strongest standards-aligned, server-agnostic A/B engine for embedded Linux and wraps cleanly behind a Helix OS-adapter, with a security model (CMS/X.509/HSM) that exceeds an ad-hoc SHA-256+signature scheme and a streaming model that fits a custom Go HTTP control plane. The cost is a self-maintained on-device D-Bus binding and bootloader integration per target. Action: carry RAUC into ota_landscape_report.md and the engine-selection ADR as the

leading Linux-phase candidate, gated by a hands-on spike (build verity bundle → stream from Helix Go control plane over HTTP Range → drive InstallBundle/Completed/Mark) and re-verification of the flagged facts.

Overall confidence: medium. Architecture, security model, streaming mechanism, D-Bus surface, and bootloader support are well-supported by primary docs; specific version/date/benchmark numbers and the Go-binding maturity need live re-confirmation, and Android relevance is firmly “none.”

§13. Sources consulted

Primary (official): - RAUC Basics — <https://rauc.readthedocs.io/en/latest/basic.html>
- RAUC Advanced Topics — <https://rauc.readthedocs.io/en/latest/advanced.html> -
Using RAUC — <https://rauc.readthedocs.io/en/latest/using.html> - RAUC GitHub
repository — <https://github.com/rauc/rauc> - RAUC project site — <https://rauc.io/> -
HTTP streaming PR #755 — <https://github.com/rauc/rauc/pull/755> - raucdbus Go
package — <https://pkg.go.dev/gitlab.com/zygoon/go-rauc/raucdbus>

Secondary (cross-checks; treated as lower confidence): - Rugix: “Comparing Open-Source OTA Update Engines for Embedded Linux” (2026-02-28) — <https://rugix.org/blog/2026-02-28-ota-update-engines-compared/> - ProteanOS: “OTA Updates in 2026: RAUC vs SWUpdate vs Mender” — <https://proteanos.com/doc/ota-updates-rauc-swupdate-mender-2026/> - FOSDEM 2023: “Delta-like Streaming of (encrypted) OTA Updates for RAUC” (slides) — https://archive.fosdem.org/2023/schedule/event/delta_like_ota_streaming/